Java Arrays — Beginner Notes

1. What is an Array?

- An **array** is a collection of multiple values of the **same data type** stored under a single variable.
- Most commonly used array operations come from the Arrays **class**: java.util.Arrays.

Instead of creating separate variables:

```
int mark1 = 80;
int mark2 = 90;
int mark3 = 70;
```

We can use an array:

```
int[] marks = {80, 90, 70};
```

Simple Definition

Array = A collection of similar data stored in a single variable and accessed using an index.

---

2. Why do we use Arrays?

- The main reason we use an **array** is to store **multiple values of the same type in a single variable**.

For example:

```
int[] marks = {80, 90, 75, 60, 88};
```

Here, all marks are stored in one array.

Advantages

- Store multiple values in one variable
- Easy to access values using an index
- Easy to process values using loops
- Useful for searching, sorting, finding maximum/minimum, etc.

---

3. Index of an Array

- Array indexing starts from **0**.

  int[] numbers = {10, 20, 30, 40};

Index	Value
0	10
1	20
2	30
3	40

  System.out.println(numbers[0]); // 10

  System.out.println(numbers[2]); // 30

Important: First element → index 0, last element → length - 1.

---

4. How to Declare an Array

- There are three commonly seen syntaxes:

  int[] numbers;

  int [] numbers;

  int numbers[];

- **Recommended**

  Use:

  int[] numbers;

  It is easier to read and commonly preferred.

---

5. How to Create an Array

- Use the new keyword:

  int[] numbers = new int[5];

- This creates an array that can store **5 integers**.

  Indexes will be:

  0 1 2 3 4

---

6. Initialize an Array

- We can assign values using indexes:

```
int[] numbers = new int[3];
numbers[0] = 10;
numbers[1] = 20;
numbers[2] = 30;
```

- Or initialize directly:

```
int[] numbers = {10, 20, 30};
```

This is the simpler approach when we already know the values.

---

7. Array Length

- We can use the length property to find the size of an array.

```
int[] numbers = {10, 20, 30, 40};
System.out.println(numbers.length);
Output:
4
```

- **Remember:** length is a property, not a method.

```
numbers.length     // Correct
numbers.length()   // Wrong
```

---

8. Traversing an Array

- Traversing means **visiting each element one by one**.
- **Using normal for loop**

```
int[] numbers = {10, 20, 30, 40};

for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
```

}

- **Using for-each loop**

for (int number : numbers) {

System.out.println(number);

}

Difference

- Use a normal for loop when you need the **index**.
- Use a for-each loop when you simply need the **values**.

---

9. Types of Arrays

Mainly, we can work with:

1. Single-Dimensional Array

A simple list of values.

int[] numbers = {10, 20, 30, 40};

2. Multi-Dimensional Array

An array containing other arrays.

Most commonly, we use a **2D array**.

int[][] numbers = {

{10, 20, 30},

{40, 50, 60}

};

Think of it like a table:

10 20 30

40 50 60

---

10. Two-Dimensional Array

- A 2D array has **rows and columns**.

int[][] numbers = new int[2][3];

This means:

- 2 rows
- 3 columns

- Access an element:

  numbers[0][1] = 20;

  Here:

  0 → row

  1 → column

- **Traversing 2D Array**

```
for (int i = 0; i < numbers.length; i++) {
    for (int j = 0; j < numbers[i].length; j++) {
        System.out.println(numbers[i][j]);
    }
}
```

---

11. Jagged Array

- A **jagged array** is a 2D array where different rows can have different numbers of columns.

```
int[][] numbers = new int[3][];

numbers[0] = new int[2];
numbers[1] = new int[4];
numbers[2] = new int[3];
```

It looks like:

10 20

30 40 50 60

70 80 90

---

12. Array Default Values

- When we create an array using new, Java automatically gives default values.

For an int array:

int[] numbers = new int[3];

Values are:

0 0 0

Some important defaults:

Data Type	Default Value
int	0
double	0.0
boolean	false
char	\u0000
Object/String	null

---

13. Array Size is Fixed

- Once an array is created, its size **cannot be changed**.

  int[] numbers = new int[3];
- This array can hold only 3 elements.
- You cannot increase it to 5 directly.
- If you need a dynamically growing collection, consider **ArrayList**.

---

14. Array Can Store Primitive or Object Types

- **Primitive values**

  int[] numbers = {10, 20, 30};
- **Objects**

  String[] names = {"Amar", "Rahul", "Priya"};

---

15. Passing Array to a Method

- An array can be passed as a method parameter.

```
static void printArray(int[] numbers) {
    for (int number : numbers) {
        System.out.println(number);
    }
}
```

Call it:

```
int[] numbers = {10, 20, 30};
printArray(numbers);
```

- The method receives a reference to the same array.

---

16. Anonymous Array

- An array can be created directly without storing it in a variable.

```
printArray(new int[]{10, 20, 30});
```

- This is called an **anonymous array**.
- It is useful when the array is needed only temporarily.

---

17. Returning an Array from a Method

- A method can also return an array.

```
static int[] getNumbers() {
    return new int[]{10, 20, 30};
}
```

Use it:

```
int[] numbers = getNumbers();
```

---

18. ArrayIndexOutOfBoundsException

- If we try to access an invalid index, Java throws:

  ArrayIndexOutOfBoundsException

  Example:

  int[] numbers = {10, 20, 30};

  System.out.println(numbers[3]);

Indexes are only:

  0 1 2

  So index 3 is invalid.

- **Important**

  For an array of size n:

  Valid indexes = 0 to n-1

---

19. Copying an Array

- Java provides System.arraycopy().

  int[] source = {10, 20, 30, 40};

  int[] destination = new int[4];

System.arraycopy(source, 0, destination, 0, 4);

It copies elements from one array to another.

---

20. Cloning an Array

- We can create a copy using clone().

  int[] numbers = {10, 20, 30};

  int[] copy = numbers.clone();

  Now numbers and copy are two different array objects.

  System.out.println(numbers == copy);

  Output:

false

---

21. Array as an Object

- In Java, an array is an **object**.
- Therefore, arrays have methods inherited from Object, such as clone().

  For example:

  ```
  int[] numbers = {10, 20, 30};


  System.out.println(numbers.getClass());
  ```

- You don't need to focus heavily on the internal JVM representation when learning arrays. The important point is:
- **In Java, arrays are objects created in memory and accessed through a reference.**

---

22. Common Array Operations

Arrays are commonly used for programs such as:

- Find largest element
- Find smallest element
- Find second largest element
- Find second smallest element
- Find sum of elements
- Find average
- Reverse an array
- Sort an array
- Search an element
- Find duplicate elements
- Remove duplicates
- Count frequency
- Find even and odd numbers

- Left rotation
- Right rotation
- Copy an array
- Compare two arrays
- Merge two arrays

⭐ **Important Points to Remember**

1. Array stores **multiple values of the same type**.
2. Array index starts from **0**.
3. Last index is **length - 1**.
4. Array size is **fixed**.
5. Use array.length to get the size.
6. Arrays can store **primitive values or objects**.
7. Java supports **1D, 2D and multidimensional arrays**.
8. A 2D array is essentially an **array of arrays**.
9. A jagged array can have different column sizes for different rows.
10. Invalid index access causes **ArrayIndexOutOfBoundsException**.
11. Arrays can be **passed to methods**.
12. Methods can **return arrays**.
13. Arrays can be copied using System.arraycopy() or clone().
14. Arrays are **objects in Java**.
